Engs 31 / CoSc 56 Final Project Report
RiSC-16 CPU

Jagger Winkler Aditya Binay

August 26, 2026

Abstract

This project implements a 16-bit RISC-16 processor on a Xilinx FPGA hosted on a Digilent Basys3 board. VHDL was primarily used to code the functionalities of the project, which was synthesized with Vivado 2024.2. The design of the processor calls for a multi-cycle architecture, where each instruction executes over a sequence of controller states, leading to a simpler datapath and shorter critical paths.

There are seven primary components in the system: a program counter, a combined instruction and data memory implemented as a Block Memory Generator IP, an instruction register, a three-port register file with a hardwired zero register, an arithmetic-logic unit (ALU) that can support operations such as addition, nand, and upper-immediate shifting, a finite-state-machine controller, and a seven-segment display driver that showcases the current value of register seven. There are four multiplexers that appropriately steer the data between components based on the controller's directions. To avoid having to resynthesize the design for each test program, a bootloader is implemented that assembles RISC-16 from a text file and transfers the resulting machine code to the board over the USB-UART link, where a receiver writes it into program memory. A single command handles environment setup, assembly, and upload, and a reset button then clears the display and begins execution.

The verification proceeded in two stages. First, behavioral simulations were performed on Vivado XSim and EDA Playground using the test benches that were written for testing each individual component. The simulations were instrumental in observing the program counter, instruction register, the ALU, the register file, and controller states. Sample inputs were passed into each component, and correct outputs were confirmed before continuing to the next stage: hardware validation. Several instructions were implemented, and the seven-segment display was used to confirm the results stored in register seven matched the expected outputs.

Contents

1 System Overview **4**
- 1.1 Top-level Block Diagram 4
- 1.2 Description of Ports 4
- 1.3 Description of Components 4
- 1.4 Supporting Software 4
  - 1.4.1 Purpose and Motivation 5

1.4.2 Usage and Workflow . . . 5
1.4.3 Input and Output Formats . . . 6

2 Technical Description **6**
2.1 Controller . . . 6
2.1.1 Description of Ports . . . 7
2.1.2 Register Transfer Level (RTL) Diagram . . . 8
2.1.3 Finite State Machine (FSM) Diagram . . . 8
2.2 ALU . . . 9
2.2.1 Description of Ports . . . 9
2.2.2 Register Transfer Level (RTL) Diagram . . . 9
2.2.3 Finite State Machine (FSM) Diagram . . . 9
2.2.4 Description of Memory . . . 9
2.3 Memory . . . 9
2.3.1 Description of Ports . . . 9
2.3.2 Register Transfer Level (RTL) Diagram . . . 10
2.3.3 Finite State Machine (FSM) Diagram . . . 10
2.3.4 Description of Memory . . . 10
2.4 Register File . . . 10
2.4.1 Description of Ports . . . 10
2.4.2 Register Transfer Level (RTL) Diagram . . . 10
2.4.3 Finite State Machine (FSM) Diagram . . . 10
2.4.4 Description of Memory . . . 10
2.5 Program Counter . . . 10
2.5.1 Description of Ports . . . 10
2.5.2 Register Transfer Level (RTL) Diagram . . . 10
2.5.3 Finite State Machine (FSM) Diagram . . . 10
2.5.4 Description of Memory . . . 10
2.6 Instruction Register . . . 10
2.6.1 Description of Ports . . . 10
2.6.2 Register Transfer Level (RTL) Diagram . . . 11
2.6.3 Finite State Machine (FSM) Diagram . . . 11
2.6.4 Description of Memory . . . 11
2.7 UART Receiver . . . 11
2.7.1 Description of Ports . . . 11
2.7.2 Register Transfer Level (RTL) Diagram . . . 11
2.7.3 Finite State Machine (FSM) Diagram . . . 12
2.7.4 Description of Memory . . . 12
2.8 Bootloader Controller and Registers . . . 12
2.8.1 Description of Ports . . . 13
2.8.2 Register Transfer Level (RTL) Diagram . . . 13
2.8.3 Finite State Machine (FSM) Diagram . . . 14
2.8.4 Description of Memory . . . 14

3 Design Validation **15**
3.1 Controller . . . 15
3.1.1 Behavioral Simulations . . . 15
3.1.2 Hardware Validation . . . 15
3.2 ALU . . . 16
3.2.1 Behavioral Simulations . . . 16
3.2.2 Hardware Validation . . . 16
3.3 Memory . . . 16
3.3.1 Behavioral Simulations . . . 16
3.3.2 Hardware Validation . . . 16
3.4 Register File . . . 16
3.4.1 Behavioral Simulations . . . 16
3.4.2 Hardware Validation . . . 16
3.5 Program Counter . . . 16
3.5.1 Behavioral Simulations . . . 16
3.5.2 Hardware Validation . . . 16
3.6 Instruction Register . . . 16
3.6.1 Behavioral Simulations . . . 16
3.6.2 Hardware Validation . . . 16
3.7 UART Receiver . . . 16
3.7.1 Behavioral Simulations . . . 16
3.7.2 Hardware Validation . . . 16
3.8 Bootloader Controller and Registers . . . 17
3.8.1 Behavioral Simulations . . . 17
3.8.2 Hardware Validation . . . 17
3.9 <Subsystem 1> . . . 17
3.9.1 <Subcomponent 1> . . . 17
3.9.2 <Subcomponent 2> . . . 17
3.10 Overall Design . . . 18
3.10.1 Behavioral Simulations . . . 18
3.10.2 Hardware Validation . . . 18

4 Analysis of the Design **18**
4.1 Resource Utilization . . . 18
4.2 Residual Warnings . . . 18
4.3 Division of Labor . . . 18
4.4 Future Work . . . 18

5 Acknowledgements **19**

6 Conclusions **19**

A VHDL Source Code **19**

B VHDL Testbenches **19**

C Support Script Source Code **19**

1 System Overview

1.1 Top-level Block Diagram

1.2 Description of Ports

Port	Dir.	Width	Description
clk	In	1	100 MHz system clock
<port>	In	<n>	<description>
<port>	Out	<n>	<description>

Table 1: Top-level ports.

1.3 Description of Components

Controller A Moore finite state machine that sequences every instruction through fetch, decode, and execute. It decodes the opcode field of the word returning from memory and drives all of the datapath's mux selects, write enables, register addresses, and the sign- or zero-extended immediate.

ALU <Short description.>

Memory <Short description.>

Register File <Short description.>

Program Counter <Short description.>

Instruction Register <Short description.>

UART Receiver An 8N1 serial receiver that recovers one byte per frame from the Basys-3 USB–UART bridge at 115200 baud, together with a byte-pair assembler that joins two consecutive bytes into one 16-bit word. It is a block inside `bootloader.vhd` rather than a separate entity.

Bootloader Controller and Registers A second Moore FSM, a payload terminal-count register, and a payload address counter. The FSM takes the first word off the UART as the number of instruction words to expect, writes each following word into consecutive memory addresses, and then asserts `Boot_Done` to hand the memory port and the CPU over to the Controller.

Top Level <Short description.>

1.4 Supporting Software

The bootloader is only half of the program-loading story: it defines a wire protocol, but something on the host side has to produce a program in that format and push it down the serial link. Three pieces of software sit outside the FPGA and make up that host-side toolchain.

`assembler.c` A two-pass RiSC-16 assembler that turns human-readable assembly into one hexadecimal machine word per line.

`uploader.py` A Python script that reads the assembler's output, frames it into the bootloader's wire protocol, finds the board's serial port, and transmits the payload.

`upload.ps1` A PowerShell driver that runs the whole flow with one command, creating the Python virtual environment and compiling the assembler the first time it is invoked. A `Makefile` provides the same two steps for a POSIX shell.

1.4.1 Purpose and Motivation

Before the bootloader existed, changing the test program meant editing a `.coe` file, regenerating the Block Memory Generator IP, and rerunning synthesis and implementation — several minutes of turnaround for a one-line change to the program. The booloader makes hardware testing much faster, and the device more practical.

The assembler is derived from the reference RiSC-16 assembler by Bruce Jacob, which is written in pre-ANSI K&R C and does not compile cleanly under a modern toolchain. It was refactored into modern C (prototyped functions, `stdlib.h`/`ctype.h` included, `strtol` instead of hand-rolled parsing) so that it builds warning-free with `gcc -Wall -Wextra`, and its output format was set to match what the uploader expects.

The uploader exists because the bootloader's protocol is specific enough that a generic terminal program cannot produce it. The payload has to be preceded by a word count, and every word has to go out high byte first; a serial monitor sending ASCII would produce neither. It also handles the practical problem of finding the board, which on Windows appears as an arbitrary `COM` number that changes between machines and USB ports.

1.4.2 Usage and Workflow

The assembler is a single C file with no dependencies beyond the standard library. The uploader is Python 3 and depends only on `pyserial`, pinned in `requirements.txt`. Neither one is touched by Vivado: the bitstream is built once, and after that the entire edit-run cycle is host-side.

The normal flow is one command:

Listing 1: Assembling and uploading a program.

```
PS> .\upload.ps1                    # program.s, autodetected COM port
PS> .\upload.ps1 -Program test.s -Port COM4
PS> .\upload.ps1 -ListPorts         # show every serial port
```

That expands into four steps:

1. **Environment setup (first run only).** `upload.ps1` creates `.venv`, installs `pyserial`, and compiles `assembler.c` with whichever of `gcc` or `cl` is on the `PATH`. Later runs recompile only if the source is newer than the executable.

2. **Assembly.** `assembler.exe program.s program.hex` produces one four-digit hex word per line.

3. **Framing.** `uploader.py` parses the hex file, prepends the word count, and serialises everything big-endian.

4. **Transmission.** The payload is written to the board's serial port at 115200 baud, 8N1. The bootloader writes the words into memory, raises `Boot_Done`, and the CPU starts executing at address 0.

The board is identified by looking for the USB–UART bridge. If exactly one is present it is used automatically; if none or several are found the script lists what it can see and exits rather than

guessing. `-port` overrides the search, `-list` prints the available ports, and `-dry-run` builds and prints the payload without opening a port, which is what makes the payload inspectable without hardware attached.

One failure mode is not obvious: Windows lets only one process hold a `COM` port, so an open Vivado Hardware Manager or serial terminal makes the upload fail with a permission error. The script catches that case specifically and says which programs to close.

1.4.3 Input and Output Formats

Assembler input. RiSC-16 assembly, one instruction per line, with optional labels in the first column and `#` comments. Alongside the eight real opcodes it accepts the standard pseudo-instructions (`nop`, `halt`, `lli`, `movi`) and the directives `.fill` and `.space`. `movi` expands to two words (`lui` then `addi`), which pass one accounts for when assigning label addresses. Branch targets may be written as labels; pass two resolves them to the PC-relative offset and rejects anything outside the −64 to 63 range the 7-bit field holds.

Assembler output, uploader input. One four-digit lowercase hex word per line. `uploader.py` strips `#` and `//` comments and blank lines, and rejects any line that is not a hex value inside 16 bits.

Source line	Encoding	Fields	Hex
`addi r1, r0, 5`	001 001 000 0000101	op, rA, rB, imm7	2405
`addi r2, r0, 7`	001 010 000 0000111	op, rA, rB, imm7	2807
`addi r7, r1, 7`	001 111 001 0000111	op, rA, rB, imm7	3c87
`add r3, r1, r2`	000 011 001 0000 010	op, rA, rB, rC	0c82

Table 2: The first four lines of a sample program and the words the assembler emits for them.

Uploader output, bootloader input. A flat byte stream: a 16-bit word count, then that many 16-bit instruction words, each sent high byte first as two 8N1 frames. The 18-word program above becomes 38 bytes beginning `00 12 24 05 28 07 3c 87 0c 82 ...`, where `0x0012` is the count of 18 and `0x2405` is the first instruction.

Listing 2: The framing step from `uploader.py`.

```
def build_payload(words):
    payload = bytearray(len(words).to_bytes(2, "big"))      # header: word count
    for w in words:
        payload += w.to_bytes(2, "big")                     # then the program
    return bytes(payload)
```

2 Technical Description

2.1 Controller

The Controller is the only piece of sequencing logic in the CPU. It holds no architectural state of its own — no PC, no registers, no accumulator — and its entire job is to walk one instruction at a time through fetch, decode, and execute while asserting exactly the control signals that instruction needs. It is written as a pure Moore machine. The implementation is three processes and two concurrent assignments: a state register, next-state logic, output logic, and two combinational formatters that build the immediate.

2.1.1 Description of Ports

Port	Dir.	Width	Description
`Clk`	In	1	100 MHz system clock
`Start`	In	1	Held in `FETCH` with all outputs at their defaults while low. Driven by `Boot_Done`, so the CPU cannot execute until the payload is in memory
`Mem_Output`	In	16	Word returning from memory. Used for dispatch during `DECODE` and as the write-back value for `lw`
`IR`	In	16	Held instruction; the source of every register address and immediate field during execute
`PC_value`	In	16	PC + 1, used as the link value for `jalr`
`ALU_Result`	In	16	ALU output, routed to the register file, memory address, or PC depending on the instruction
`Reg_A`	In	16	Register file port A, the value stored by `sw`
`EQ_flag`	In	1	Comparator output; drives `PC_Sel` during `BEQ`
`PC_Sel`	Out	1	0 selects PC + 1, 1 selects the branch or jump target
`PC_En`	Out	1	Asserted in the state where the instruction retires
`PC_Write`	Out	16	Branch or jump target, taken from `ALU_Result`
`Mem_Set_Addr`	Out	16	Data address for `lw`/`sw`, taken from `ALU_Result`
`Mem_Set_Addr_en`	Out	1	0 selects the PC path into the address mux (instruction fetch), 1 selects the data address
`Mem_write`	Out	16	Store data, taken from `Reg_A`
`Mem_write_en`	Out	1	Memory write enable; asserted only in `SW`
`IR_En`	Out	1	Holds the fetched word into the instruction register; asserted only in `DECODE`
`Reg_Addr_A`	Out	3	Read port A address (`sw` source, `beq` operand)
`Reg_Addr_B`	Out	3	Read port B address, the first ALU operand
`Reg_Addr_C`	Out	3	Read port C address, the second ALU operand for RRR-type
`Reg_Addr`	Out	3	Write port address, always the rA field
`Reg_write`	Out	16	Write data: `ALU_Result`, `Mem_Output`, or `PC_value`
`Reg_w_en`	Out	1	Register file write enable
`Opcode`	Out	3	Passed to the ALU; `IR(15 downto 13)`
`Immediate`	Out	16	Formatted immediate, sign- or zero-extended as the instruction requires
`Read_Immediate`	Out	1	1 selects `Immediate` as the ALU's second operand, 0 selects `Q_C`
`Read_PC`	Out	1	1 selects PC + 1 as the ALU's first operand, used only by `beq`

Table 3: Controller ports.

2.1.2 Register Transfer Level (RTL) Diagram

Structurally the Controller is almost all control logic, so its RTL diagram is dominated by the state register and the two decode paths that fan out of it. The only datapath elements are the two immediate formatters, both purely combinational:

- `Imm_Sign` sign-extends `IR(6 downto 0)` to 16 bits. RRI-type immediates (`addi`, `lw`, `sw`, `beq`) are signed, covering −64 to 63, and the ALU adds them as signed values.
- `Imm_Wide` zero-extends `IR(9 downto 0)` to 16 bits for `lui`. The shift-by-six is done inside the ALU, not here, so the Controller passes the raw ten-bit field rather than a pre-shifted value.

The next-state logic dispatches on `Mem_Output(15 downto 13)` rather than on `IR(15 downto 13)`. The instruction register is loaded on the clock edge leaving `DECODE`, so `IR` still holds the *previous* instruction while the dispatch decision is being made. Decoding the memory output directly avoids adding a state purely to wait for `IR` to settle. Once the machine is in an execute state, `IR` is valid and every register address and immediate field is taken from it.

Second, the link value for `jalr` is taken from `PC_value` rather than being computed in the ALU. That leaves the ALU free to compute the jump target as `Q_B + r0` in the same state, so `jalr` retires in one execute state instead of two. Because the register file reads asynchronously, the degenerate case `jalr rA, rA` still works: `Q_B` is captured before the write edge.

2.1.3 Finite State Machine (FSM) Diagram

The Controller is an eleven-state Moore machine. The state register is synchronous and resets to `FETCH` whenever `Start` is low, which is how the bootloader holds the CPU off until the program is in memory.

`FETCH` All outputs at their defaults. With `Mem_Set_Addr_en` low the address mux is on the PC path, so the memory sees the program counter and registers that address on the edge out of this state.

`FETCH2` Also silent. This state exists solely to cover the one-cycle read latency of the synchronous block RAM: the address is captured on the edge into `FETCH2`, and the instruction word is valid on `Mem_Output` for the whole of the following state, which is what the dispatch logic needs.

`DECODE` Asserts `IR_En`, storing the fetched word. The next state is chosen from `Mem_Output(15 downto 13)`: `000` and `010` both go to `RRR`, `001` to `ADDI`, `011` to `LUI`, `100` to `SW`, `101` to `LW`, `110` to `BEQ`, and `111` to `JALR`.

`RRR` Serves both `add` and `nand`, which differ only in the opcode handed to the ALU. Reads rB and rC, writes `ALU_Result` to rA, and advances the PC.

`ADDI` Reads rB, selects the sign-extended immediate, writes `ALU_Result` to rA, and advances the PC.

`LUI` Selects the wide immediate and writes the shifted result to rA. No register is read; the ALU ignores its first operand for this opcode.

`SW` Drives the ALU result onto the memory address port and `Reg_A` onto the write data port with `Mem_write_en` asserted, then advances the PC. One state is enough because a write does not need the data to come back.

`LW` Presents the address only. `PC_En` and `Reg_w_en` both stay low, because the loaded word has not arrived yet.

`LOAD_A` The second half of `lw`. The data address is still driven and still selected — releasing it here would point the memory back at the PC — and `Mem_Output` is written into rA. The PC advances and the instruction retires.

`BEQ` Reads rA and rB, selects PC + 1 and the sign-extended immediate into the ALU so the target is computed as PC + 1 + offset, and drives `PC_Sel` directly from `EQ_flag`. The comparator hangs off the register file's A and B ports and does not contend with the ALU, so the compare and the branch fit in one state. This is the longest combinational path in the design and is the first place to look if timing closure ever becomes a problem.

`JALR` Writes `PC_value` to rA as the link, computes the target in the ALU, and forces `PC_Sel` high because the jump is unconditional.

Every execute state returns to `FETCH`, except `LW`, which goes to `LOAD_A` first. That gives four clock cycles for every instruction and five for `lw` — 40 ns and 50 ns respectively at 100 MHz.

2.2 ALU

2.2.1 Description of Ports

Port	Dir.	Width	Description
<port>	<In/Out>	<n>	<description>

Table 4: <Component 2> ports.

2.2.2 Register Transfer Level (RTL) Diagram

2.2.3 Finite State Machine (FSM) Diagram

2.2.4 Description of Memory

2.3 Memory

2.3.1 Description of Ports

Port	Dir.	Width	Description
<port>	<In/Out>	<n>	<description>

Table 5: <Component 3> ports.

2.3.2 Register Transfer Level (RTL) Diagram

2.3.3 Finite State Machine (FSM) Diagram

2.3.4 Description of Memory

2.4 Register File

2.4.1 Description of Ports

Port	Dir.	Width	Description
<port>	<In/Out>	<n>	<description>

Table 6: <Component 4> ports.

2.4.2 Register Transfer Level (RTL) Diagram

2.4.3 Finite State Machine (FSM) Diagram

2.4.4 Description of Memory

2.5 Program Counter

2.5.1 Description of Ports

Port	Dir.	Width	Description
<port>	<In/Out>	<n>	<description>

Table 7: <Component 5> ports.

2.5.2 Register Transfer Level (RTL) Diagram

2.5.3 Finite State Machine (FSM) Diagram

2.5.4 Description of Memory

2.6 Instruction Register

2.6.1 Description of Ports

Port	Dir.	Width	Description
<port>	<In/Out>	<n>	<description>

Table 8: <Component 6> ports.

2.6.2 Register Transfer Level (RTL) Diagram

2.6.3 Finite State Machine (FSM) Diagram

2.6.4 Description of Memory

2.7 UART Receiver

The UART receiver recovers bytes from the Basys-3's USB–UART bridge and assembles them into the 16-bit words the rest of the bootloader works in. It handles three separate problems: crossing from the asynchronous serial line into the 100 MHz clock domain, recovering bit timing from a line that carries no clock, and pairing bytes into words.

It is a block within `bootloader.vhd` rather than a standalone entity. The two are described separately here because they are separate blocks on the bootloader's RTL diagram and were developed and tested as separate pieces, but they compile as one design unit. The signals listed below are therefore internal signals, not entity ports; only `RsRx` crosses the entity boundary.

2.7.1 Description of Ports

Signal	Dir.	Width	Description
`clk`	In	1	100 MHz system clock
`reset`	In	1	Synchronous, active high; returns the receiver to `ST_IDLE`
`RsRx`	In	1	Raw serial line from the USB–UART bridge. Idles high; asynchronous to `clk`
`rx_bit`	Int	1	`RsRx` after two-stage synchronisation; the value the receiver actually samples
`rx_byte`	Out	8	The most recently received byte, valid when `byte_valid` is high
`byte_valid`	Out	1	One-clock pulse, asserted once per correctly framed byte
`rx_data`	Out	16	Assembled word, high byte first. Feeds both the payload terminal count register and the memory write data port
`word_valid`	Out	1	One-clock pulse per complete word. This is the "`Packet_Ready`" signal on the RTL diagram

Table 9: UART receiver signals. `Int` marks an internal signal that is neither an input nor an output of the block.

2.7.2 Register Transfer Level (RTL) Diagram

The receiver is built from four pieces in series: a synchroniser, a bit-timing counter, a shift register, and a byte-pair assembler.

Synchroniser. `RsRx` arrives from an off-chip bridge with no relationship to the system clock, so sampling it directly risks metastability on the flip-flops that consume it. A two-stage shift register (`rx_sync`) absorbs that: stage 0 is the fresh sample and is the one allowed to go metastable, and stage 1 is the clean value everything downstream uses.

Bit timing. At 115200 baud on a 100 MHz clock, `CLKS_PER_BIT` = 100,000,000/115,200 = 868 and `HALF_BIT` = 434. Both are computed from the `CLK_FREQ_HZ` and `BAUD_RATE` generics, The

integer division truncates, giving an accumulated error of about 0.06% over a ten-bit frame, far within the margin of error of the framing.

Shift register. Data bits are shifted into `byte_reg` from the top, since 8N1 sends the least significant bit first. The byte is transferred to `rx_byte` and `byte_valid` is pulsed only if the stop bit is found high; a frame with a bad stop bit is discarded silently rather than handed on as corrupt data.

Byte-pair assembler. A single flag, `byte_phase`, tracks whether the next byte is the high or the low half of a word. The first byte is stored into `high_byte`; the second is concatenated with it into `rx_data` and `word_valid` is pulsed. High byte first matches the big-endian framing in `uploader.py`.

Both `byte_valid` and `word_valid` are generated by assigning `'0'` as the first statement in their clocked process and overwriting it only on the cycle the event occurs. That produces a clean one-clock pulse in the `clk` domain, which matters: it means the bootloader FSM can treat the signal as a level and never has to edge-detect something that stays high for many cycles.

2.7.3 Finite State Machine (FSM) Diagram

The receiver is a four-state Moore machine.

`ST_IDLE` Counters cleared, waiting for the line to fall. A high-to-low transition on `rx_bit` is a possible start bit and moves the machine to `ST_START`.

`ST_START` Counts to `HALF_BIT` and re-checks the line. If it is still low the edge was a real start bit and the machine proceeds to `ST_BITS`; if it has gone high the edge was noise and the machine returns to `ST_IDLE`. Landing on the midpoint here is what puts every later sample in the centre of its bit.

`ST_BITS` Waits a full `CLKS_PER_BIT` between samples, shifting each one into `byte_reg`. After eight bits (`bit_index` reaching 7) it moves to `ST_STOP`.

`ST_STOP` Waits one more bit period and checks for a high stop bit. If the framing is good, `rx_byte` is updated and `byte_valid` pulses. Either way the machine returns to `ST_IDLE`, ready for the next start bit.

2.7.4 Description of Memory

The receiver contains no addressed memory. It holds two 8-bit registers — `byte_reg`, the shift register the incoming frame is assembled in, and `high_byte`, which holds the first byte of a pair while the second arrives — plus the 16-bit `rx_data` output register. There is no FIFO: a word must be consumed by the bootloader FSM before the next one completes. At 115200 baud a byte takes roughly 87 μs to arrive and the FSM needs two clock cycles to write a word, so the margin is about four orders of magnitude. A FIFO would be needed only if the FSM were to stall for tens of milliseconds, in which case we have *much* bigger problems.

2.8 Bootloader Controller and Registers

The bootloader is what makes the program a piece of data rather than a piece of the bitstream. Without it, changing a single instruction means regenerating the memory IP and rerunning implementation; with it, the same bitstream runs any program that fits in memory, loaded in well under a second.

It owns the memory write port from power-up until the payload is complete, then releases it and starts the CPU. Three blocks sit downstream of the UART receiver: a Moore FSM, a terminal-count register holding the expected word count, and a counter that doubles as the write address.

2.8.1 Description of Ports

These are the ports of the `Bootloader` entity, which encloses both this block and the UART receiver of Section 2.7.

Port	Dir.	Width	Description
`clk`	In	1	100 MHz system clock
`reset`	In	1	Synchronous, active high. Wired to btnC (U18); clears the FSM, both registers, and the UART receiver so a new program can be loaded without reprogramming the board
`RsRx`	In	1	USB–UART receive line, pin B18. Idles high
`Boot_Addr`	Out	16	Write address, `payload_ct` + `BASE_ADDR`
`Boot_Data`	Out	16	Write data, driven straight from `rx_data`
`Boot_We`	Out	1	Write enable, high for exactly one clock per word
`Boot_Done`	Out	1	Goes high once the whole payload is in memory and stays high. Switches the boot mux over to the CPU and releases the Controller's `Start`
`Boot_Busy`	Out	1	High while a load is in progress; available for an LED or debug pin

Table 10: Bootloader entity ports.

Generic	Default	Description
`CLK_FREQ_HZ`	`100_000_000`	System clock frequency, used with `BAUD_RATE` to derive the bit period
`BAUD_RATE`	`115_200`	Must match the `-baud` setting in `uploader.py`
`BASE_ADDR`	`0`	First memory address the payload lands on. Zero, so execution starts at the first word loaded

Table 11: Bootloader generics.

2.8.2 Register Transfer Level (RTL) Diagram

Three registers and a comparator sit between the UART receiver and the memory write port.

Payload TC (`payload_tc`, 16 bits) stores the first word off the UART — the number of instruction words to expect. It loads on `tc_en`, which the FSM asserts for exactly one cycle while `rx_data` still holds the size word.

Payload Counter (`payload_ct`, 16 bits) counts words written. It is cleared by `count_clr` and incremented by `count_en`. It serves double duty as the write address, which is what keeps the design from needing a separate address register: added to `BASE_ADDR`, it drives `Boot_Addr` directly, so word n necessarily lands at `BASE_ADDR` $+ n$ and the address can never drift out of step with the count.

Comparator produces `count_done` when `payload_ct = payload_tc`. This is the only condition that ends the load. Because the comparison is for equality rather than "greater than or equal", a payload longer than its declared count is simply ignored — the FSM has already reached `S_DONE` and stops writing — which is the behaviour the testbench checks with a trailing `0xDEAD`.

`Boot_Data` is wired straight from `rx_data` with no register in between. The word is stable from the cycle `word_valid` pulses until the next word completes, tens of thousands of cycles later, so there is nothing for an extra register to buy.

The one signal that leaves this block and changes the rest of the machine is `Boot_Done`. In the top level it selects the boot mux, gates `IR_En` and `Reg_w_en`, and drives the Controller's `Start`. While it is low the PC is force-fed zero every clock, so the PC is already sitting at address 0 the instant the CPU is released and the program counter needs no reset port of its own.

2.8.3 Finite State Machine (FSM) Diagram

A six-state Moore machine drives the three control signals plus the done flag.

`S_IDLE` Everything quiet, waiting for the first `word_valid`. `Boot_Done` is low, so the CPU is held off.

`S_GET_SIZE` One cycle with `tc_en` asserted, storing the size word. Unconditionally advances.

`S_ARM` Asserts `count_clr`, zeroing the counter. If `count_done` is already true — a declared size of zero — it goes straight to `S_DONE`; otherwise it waits for the first data word.

`S_WRITE` One cycle with `w_en` and `count_en` both asserted. The word is written and the counter advances on the way out, so the address is already correct for the next word.

`S_WAIT` Checks `count_done` first: if the payload is complete the machine finishes, otherwise it waits for the next `word_valid` and returns to `S_WRITE`. This is where the machine spends nearly all of its time, since a word takes about 174 μs to arrive and the write takes two clock cycles.

`S_DONE` Asserts `done_flag`. There is no path out except `reset`: memory belongs to the CPU from here on, and letting the bootloader reclaim it mid-execution would corrupt a running program.

Splitting `S_WRITE` and `S_WAIT` rather than writing from a single state is what guarantees `Boot_We` is exactly one clock wide. Since `word_valid` is itself a one-clock pulse, the machine can only leave `S_WAIT` once per word, and a word can never be written twice.

2.8.4 Description of Memory

The bootloader contains no memory of its own; it drives the write port of the memory described in Section 2.3. That memory is a single-port block RAM, 16 bits wide and 65536 words deep, generated by the Block Memory Generator IP. The bootloader writes to it through `Boot_Addr`, `Boot_Data`, and `Boot_We`, all three of which are switched over to the CPU together by the boot mux when `Boot_Done` rises, so there is never a cycle in which half the port belongs to each.

Its own state is three registers: `payload_tc` (16 bits, the expected word count), `payload_ct` (16 bits, words written so far and the current write address), and the state register itself. The 16-bit count field is what bounds a payload to 65535 words, which matches the depth of the memory.

3 Design Validation

3.1 Controller

The Controller has no data output to check, so "correct" cannot be judged by looking at a result. What matters is whether, in each state, the right control lines are asserted and every other one is not. The testbench in `controller_tb.vhd` therefore checks both halves of that: for every instruction it verifies the signals that instruction should assert, and verifies that the signals it should not assert are low.

Two techniques make the checks meaningful rather than merely non-zero. First, each of the four data inputs is held at a distinctive constant — `ALU_Result` at `0x1234`, `Reg_A` at `0xA5A5`, `PC_value` at `0x0100`, and the loaded word at `0xBEEF`. A check on `Reg_write` then proves *which* source the output is wired to, so a mux select that is inverted or a data path that is crossed shows up as a failure rather than passing by coincidence. Second, `IR` is deliberately left holding the previous instruction during `DECODE`, so a Controller that dispatched on `IR` instead of `Mem_Output` would branch to the wrong execute state and fail the checks that follow.

Ten cases cover all eight opcodes and the field-formatting edge cases: `add` and `nand` (which share the `RRR` state and should differ only in opcode and register fields), `addi` with both a negative and a positive immediate (checking that −3 sign-extends to `0xFFFD` and +5 to `0x0005`), `lui` (checking that the immediate arrives zero-extended as `0x0155` and *not* pre-shifted to `0x5540`, since the shift belongs to the ALU), `beq` taken and not taken, `jalr` (checking that the link comes from `PC_value` while the target comes from the ALU, and that `Reg_Addr_C` is forced to `r0`), `sw`, and `lw` across both of its states. Failures are counted and reported rather than aborting the run, so one broken case does not hide the rest, and the run ends with an explicit pass or fail message.

Success criteria: zero reported errors, and the state sequence observed in the waveform matches the state diagram for every instruction — four states for each instruction except `lw`, which takes five.

3.1.1 Behavioral Simulations

3.1.2 Hardware Validation

The Controller is not separately observable on the board — it has no output pins and every one of its control signals is internal — so it is validated indirectly. Running `program.s`, which exercises all eight opcodes and leaves a known value in r7 after each group, confirms the Controller sequenced each instruction correctly, since a wrong control signal produces a wrong value on the display. Section 3.4 covers that test.

3.2 ALU

3.2.1 Behavioral Simulations

3.2.2 Hardware Validation

3.3 Memory

3.3.1 Behavioral Simulations

3.3.2 Hardware Validation

3.4 Register File

3.4.1 Behavioral Simulations

3.4.2 Hardware Validation

3.5 Program Counter

3.5.1 Behavioral Simulations

3.5.2 Hardware Validation

3.6 Instruction Register

3.6.1 Behavioral Simulations

3.6.2 Hardware Validation

3.7 UART Receiver

The UART receiver and the bootloader FSM compile as one entity, so they share a testbench (`bootloader_tb.vhd`) and are tested through the same stimulus. The receiver is nonetheless verified on its own terms first, by watching its internal signals rather than the memory write port.

Testing a UART in simulation at its real baud rate is impractical: at 115200 baud one byte takes 868 clock cycles per bit, so a short program would need tens of milliseconds of simulated time. The bit period is derived from the `CLK_FREQ_HZ` and `BAUD_RATE` generics rather than hard-coded, which makes simulation practical.

The stimulus process drives `RsRx` directly with a `send_byte` procedure that reproduces a real 8N1 frame: a low start bit, eight data bits least significant first, a high stop bit, and an idle period between frames. A `send_word` wrapper sends the high byte then the low byte, matching the order `uploader.py`.

3.7.1 Behavioral Simulations

3.7.2 Hardware Validation

The receiver is validated on hardware by the fact that the upload works at all: a program that assembles correctly on the host and produces the right values on the seven-segment display after loading is direct evidence that every byte crossed the link and was reassembled in the right order. A single dropped or misframed byte would shift every subsequent word by one byte and produce garbage instructions.

3.8 Bootloader Controller and Registers

The bootloader is checked at its memory write port, since that is its entire externally visible effect. The testbench sends a complete payload and monitors `Boot_We`, asserting on every write that the address is the one expected — a counter in the monitor process increments independently, so an address that skips, repeats, or restarts fails immediately rather than being noticed later as a wrong program.

The payload is deliberately constructed to catch the two failure modes this design is most exposed to. It declares four words, sends four distinguishable words (`0x1111` through `0x4444`), and then sends a fifth, `0xDEAD`, which must be ignored. That last word tests the boundary condition directly: if `count_done` were a "greater than" comparison, or if `S_DONE` had a path back into `S_WRITE`, the extra word would be written and the write count would come out at five.

Success criteria, all asserted at the end of the run: exactly four writes occur; they land at addresses 0 through 3 in order; each carries the word that was actually sent; the trailing word is ignored; and `Boot_Done` is high once the payload is complete.

3.8.1 Behavioral Simulations

3.8.2 Hardware Validation

On hardware the load is visible on the seven-segment display without any test equipment: while `Boot_Done` is low the display shows `Boot_Addr`, so the word count climbs as the payload arrives and then freezes at the final address, at which point the display switches to register 7 and the CPU begins executing. That single behaviour confirms three things at once — that words are arriving, that the address counter is advancing one per word, and that `Boot_Done` rose and handed the memory over.

`uploader.py` reports the number of words and bytes it sent, so the value the display freezes at can be compared against what the host claims to have transmitted. A mismatch localises the fault immediately: agreement with a program that then misbehaves points at the CPU rather than the load, while a short count points at the link or the receiver.

Pressing btnC (U18) resets the bootloader and the register file, so a second program can be uploaded without reprogramming the FPGA. Loading two different programs in succession and getting the correct result from each is the strongest single check that the reset path is complete and that no state survives from one load to the next.

3.9 <Subsystem 1>

3.9.1 <Subcomponent 1>

Behavioral Simulations

Hardware Validation

3.9.2 <Subcomponent 2>

Behavioral Simulations

Hardware Validation

3.10 Overall Design

3.10.1 Behavioral Simulations

3.10.2 Hardware Validation

4 Analysis of the Design

4.1 Resource Utilization

Resource	Used	Available	Utilization
Slice LUTs	<n>	20800	<%>
Slice Registers	<n>	41600	<%>
Block RAM Tile	<n>	50	<%>
DSPs	<n>	90	<%>
Bonded IOB	<n>	106	<%>
BUFGCTRL	<n>	32	<%>

Table 12: Post-implementation resource utilization (Artix-7 XC7A35T, Basys3).

4.2 Residual Warnings

Warning	Justification
<warning text>	<why it is safe to ignore>

Table 13: Residual warnings.

4.3 Division of Labor

Jagger was responsible for the CPU control FSM, the bootloader FSM, and the UART receiver He also wrote the assembler and the uploader script, alongside writing the top level file.

4.4 Future Work

The next natural extension of this project would be to implement pipelining. This would likely require moving from a FSM centric design to a more sequential logic design. This rework would be a fundamental rewrite of the project, but would allow for a much higher clock frequency and throughput. The difficulty and complexity of the project would also skyrocket. Other considerations for improvement would be error checking on the UART receiver, and a more robust bootloader protocol. The current bootloader protocol is very simple and does not have any error checking or correction. This could be improved by adding a checksum or CRC to the payload, and having the bootloader verify it before writing to memory.

5 Acknowledgements

Huge thank you Bruce Jacob for creating the RiSC-16 architecture and providing the original assembler. We used every resourc here. Tad was also very helpful in providing guidance and advice early on, steering us away from a complex datapath design.

6 Conclusions

This project was an absolute monster to complete in the 2 weeks we had. I (Jagger) am very happy we chose such an ambitious project, and I am very proud of the final result. The bootloader and UART receiver were a lot of fun to implement, and I learned a lot about FSM design and VHDL in general. I also have a much better understanding of the basics of how a CPU works, and how RISC architectures work in general.

A VHDL Source Code

- `controller.vhd` — the CPU control FSM
- `bootloader.vhd` — UART receiver, byte-pair assembler, payload registers, and the load FSM
- `<file>.vhd` — <short description>
- `<file>.vhd` — <short description>

B VHDL Testbenches

- `controller_tb.vhd` — self-checking testbench covering all eight opcodes and both `lw` states
- `bootloader_tb.vhd` — self-checking testbench driving a full UART payload and monitoring the memory write port
- `<file>_tb.vhd` — <short description>
- `<file>_tb.vhd` — <short description>

C Support Script Source Code

- `assembler/assembler.c` — two-pass RiSC-16 assembler
- `host/uploader.py` — serial uploader; frames the hex file into the bootloader's wire protocol
- `upload.ps1` — PowerShell driver for the full build–assemble–upload flow
- `Makefile` — the same flow for a POSIX shell
- `requirements.txt` — Python dependency (`pyserial`)
- `program.s` — test program exercising all eight opcodes